HTTP Status Quest

300

While searching the guards room, you find a purse. To check if it is full of gold or totally empty, roll 1 die. If you roll a 1-2, turn to 200. If you roll a 3-4 turn to 204. If you roll a 5-6 turn to 404.

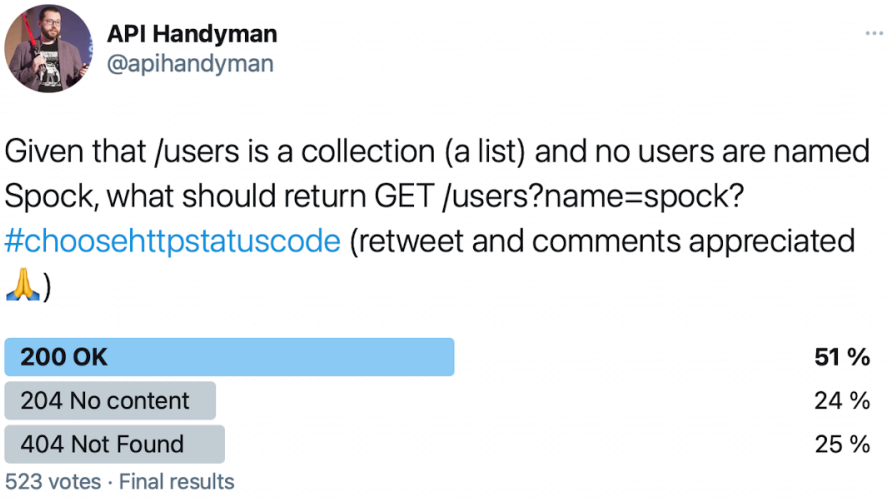
Choosing HTTP status codes Series - Part 4

Empty list - HTTP status code

200 vs 204 vs 404

The context

When you need to represent lists (aka collection resources) in a REST/RESTful/RESTish API, a usual design pattern is to represented with them a `/resources` (or `/resource`, read /resources rules and /resource sucks … or is it the other way around?). More often than not, you'll need to be able to return a subset of the list's elements. To do so, a usual (if not standard) design pattern is to add query parameters to provide search filters. If a `GET /users` is supposed to return a list containing all (actually accessible to the consumer and possibly to the end user) users, a `GET /users?name=spock` is supposed to return a list containing only the users whose name is `spock`. The question we will answer today basically is: which HTTP status code respond with when returning an empty list.



According to my Twitter pool, 51% of respondents would return a `200 OK`, while 24% would return a `204 No Content` and 25% would return a `404 Not Found`. Let's see what could be the correct answers according to RFC(s) and common practice.

The obvious 200 OK

> The 200 (OK) status code indicates that the request has succeeded.
> — *RFC 7231 Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content, section 6.3.1*

Let's start with the most common and valid response in such a case: `200 OK`. When responding to `GET /users`, the API will respond with that HTTP status code along with a list of all (actually accessible to the consumer and possibly to the end user) users users. When

responding to `GET /users?name=smith`, the API will respond also that with HTTP status code along with a list contains all users named `smith`. And finally when responding to `GET /users?name=spock` and if there are no user name `spock`, the API will respond yet another time with that HTTP status code but this time along with an empty list. That is actually the most common response I have ever seen, probably because most people consider that the `/users` collection/list resource exists and `name` query param is used to filter the content of the list.

But there is a more specific HTTP status code that could do the trick too.

The not so current 204

> The 204 (No Content) status code indicates that the server has successfully fulfilled the request and that there is no additional content to send in the response payload body.
>
> — *RFC 7231 Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content, section 6.3.5*

While `200 OK` being a valid and the most common answer, returning a `204 No Content` could make sense as there is absolutely nothing to return. It is indeed more often used when responding to a `PUT` (replace) or a `PATCH` (partial update), when servers don't want to bother returning the replaced/updated resource or on a `DELETE` because there is usually nothing to return after a deletion. But it can be used on a `GET` too. If the request is valid, has been successfully fulfilled and if there is no additional content to send (which is the case as the returned list would be empty), `204 No Content` is perfectly understandable and valid answer.

It's a valid response but I personally will not use it and do not recommend to use it in that case in my context. Because it is not that common (based on my experience, it's not absolute truth) and may imply more work. Actually, I try to avoid using specific/uncommon HTTP status when a more generic/common one works too, that usually simplifies consumer's job and also designer's one as there are less possible choices and behaviors (I'll write a post about that). Though a consumer must be able to interpret any `2xx` as as success and fallback to treat it as `200 OK`, that means there will be no response body, so no empty list. That could easily lead to to possible "null pointer exception" or any equivalent requiring more controls in code. A `200 OK` with an empty list can be treated the same as non empty list without thinking about it. Note that, consumer may obviously have to check il the list is empty or not to possibly show a message to end user, but the exact same code will work without that control.

But while simplifying choices, note that using that "simplified HTTP" stance, you'll loose some "HTTP semantic out of the box". Indeed tne major argument in favor of `204 No Content` is that is allows to check empty search results (especially in logs) vs non empty ones without relying on specific response body's semantic. That's quite an interesting feature. Maybe we need more APIs actually fully using HTTP semantic to make this `204 No Content` response more common and a no brainer.

So choosing between `200 OK` and `204 No Content`, depends on you and your context.

The not recommended 404

> The 404 (Not Found) status code indicates that the origin server did not find a current representation for the target resource or is not willing to disclose that one exists.
> — *RFC 7231 Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content, section 6.5.4*

I just realized that's the fourth post in this series and `404 Not Found` has been involved in all posts so far. Let's see what say the HTTP RFCs (with an s) about using it in that use case.

If we look at this status code definition in RFC 7231 and if we consider that `/users` is the resource used even when doing a `GET /users?name=spock`, returning that HTTP status code makes no sense at all because the resource `/users` exists, it's just that the list it contains may be empty.

But is this definition of a resource identifier (excluding query parameters) is actually the correct one? Section 2 of RFC 7231 states *each resource is identified by a Uniform Resource Identifier (URI), as described in Section 2.7 of RFC 7230*. That section 2.7 of RFC 7230 says the "query" (what is between the first `?` and `#`) is a part of the resource identifier. If we follow the link (it's quite a maze!) conducting to complete description of the query, we eventually arrive at Section 3.4 of RFC 3986 that says *the query component contains non-hierarchical data that, along with data in the path component (Section 3.3), serves to identify a resource*. That basically means that `/users?name=spock` is a resource identifier, so returning 404 is valid according to HTTP RFCs if we want to say "sorry no resource match the strict identifier provided in your query" or "there is no such a users list containing users named spock". But using that HTTP status code being valid from a pure HTTP perspective, is it actually a good idea to use it in that use case?

In my humble opinion, based on my experience designing APIs, reading and listening to many API practitioners, analyzing many APIs and doing hundreds of API design reviews, I do not recommend to use it in that case because that would break a common practice. In most REST/RESTful/RESTish APIs, a "resource identifier" is actually the resource path without the query part, that may be wrong when speaking strictly HTTP but that is the current state of common practice. In most APIs, `404 Not Found` is strongly attached to "there is nothing for the requested path (excluding query parameters)". It is returned in case involving `/path-that-does-not-exist` or `/collection/{id that does not exist}` (see Choosing HTTP status codes Part 2 - Hands off that resource, HTTP status code 401 vs 403 vs 404 or Choosing HTTP status codes Part 3 - Move along, no resource to see here (truly), HTTP status code 204 vs 403 vs 404 vs 410) but not for empty lists (that's usually a `2xx Success class`). Also, returning a `4xx Client Error Class` says that consumer made an error, is that really the case here? I don't think so, the consumer just provided search filters that don't match any element in a list.

That's my reasoned opinion of not using `404 Not Found` for empty lists, but if you have valid reasons to use this HTTP status code for this use case, don't forget to be consistent and provide informative error feedback. Indeed, if we take for granted that `GET /users?name=spock` returns a `404 Not Found` if there are no users named spock. What about `GET /users` if there are no users at all? It should return the same HTTP status code. And differentiating this it from a more common `/path-that-does-not-exist` will require to add some information in the response body to explain the actual cause of this response.

DX above all

The lesson of today is that following the HTTP protocol is one thing but there are sometimes various options with pros and cons and sometimes being overly strict can lead to design that are less easy to understand. The question is not about achieving the most perfect design (regarding HTTP) but just achieve a design that makes sense for most people involved and proposes the best as possible DX. And that D in DX includes developers but also designers. Simple design rules that makes sense for most are a key factor in your APIs success.